Project Planning Phase

Solution Architecture

Date	08 March 2026
Team ID	SWTID-2026-2148
Project Name	SpendWise
Maximum Marks	5 Marks

Solution Architecture:

The solution architecture for the **SpendWise** application is designed to be simple, scalable, and beginner-friendly, helping learners understand concepts such as JWT authentication, REST API development, and MongoDB integration. The system is built using **Node.js**, **Express**, and **MongoDB**, following a clean and modular backend structure for better maintainability.

SpendWise uses a **client–server architecture**, where the client (such as Postman, Thunder Client, or an optional frontend interface) sends HTTP requests to the backend APIs. The backend processes these requests by handling user authentication, executing business logic, and performing database operations. **MongoDB** is used as the database to store and manage user information and expense records efficiently.

Goals of the Solution Architecture:

Determine the Most Suitable Technology Stack:

- Leverage Node.js and Express.js to create RESTful APIs.
- Utilize MongoDB with Mongoose to manage data storage and database operations.
- Implement JWT (JSON Web Token) for robust user authentication and authorization.
- Employ Postman or Thunder Client to test and showcase API functionality.

Define Structure & Characteristics:

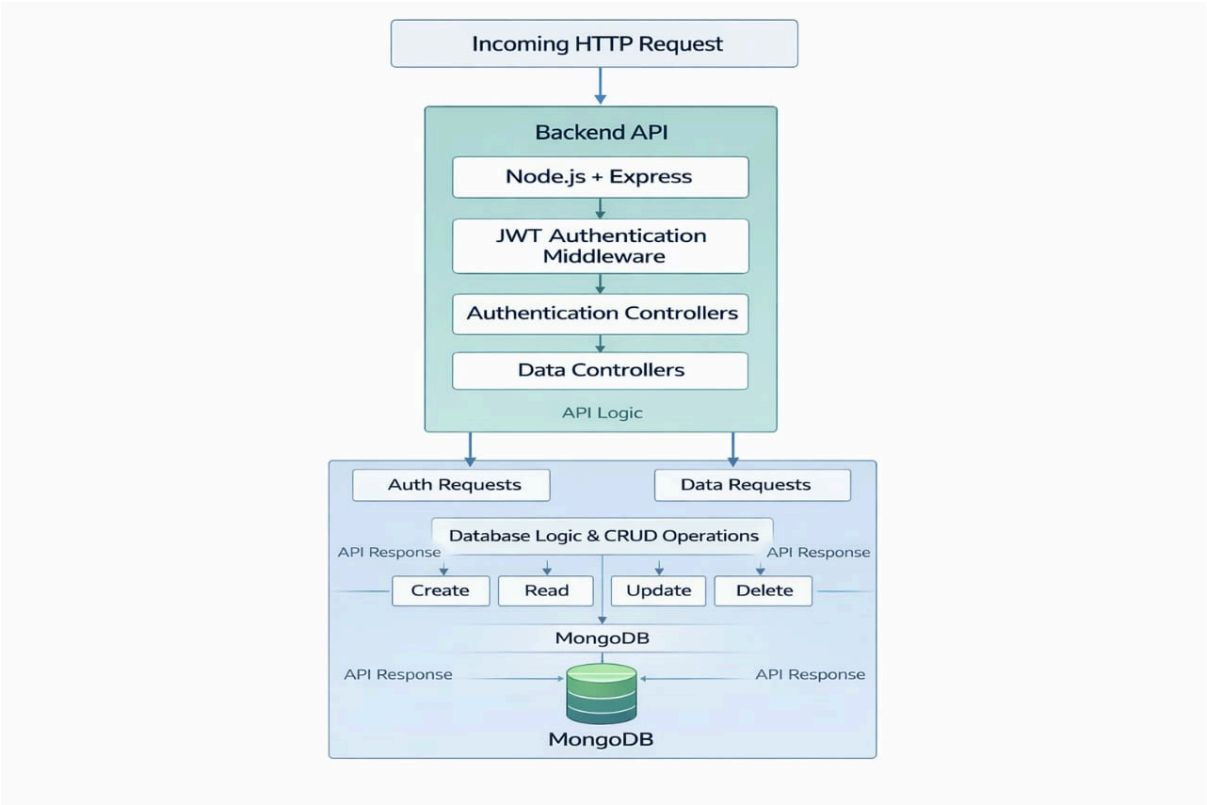
- Organize the project using a modular structure with dedicated folders for:
    - **Controllers** – handles business logic
    - **Models** – defines database schemas
    - **Routes** – manages API endpoints
    - **Middleware** – handles JWT authentication
- Ensure the architecture is:
    - Easy for beginners to understand
    - Maintainable for future updates
    - Scalable to incorporate additional features later (like filters, frontend integration, etc)

Outline Features & Development Phases:

- **Phase 1:** Initialize the project and establish MongoDB connection
- **Phase 2:** Implement user authentication (Register & Login using JWT)
- **Phase 3:** Create JWT middleware to protect specific routes
- **Phase 4:** Implement CRUD operations (Create, Read, Update, Delete) for user data
- **Phase 5:** Test APIs with Postman/Thunder Client and prepare documentation

Establish Specifications for Development & Delivery:

- **HTTP Request** enters the backend API built with **Node.js + Express**.
- The request is processed by **JWT Middleware** for authentication.
- Then it is routed to either **Auth Controllers** or **Record Controllers**.
- Both controllers perform **CRUD operations** (Create, Read, Update, Delete).
- These operations interact with **MongoDB**.

- The API response is sent back to the client.

STRUCTURE: